

A Novel Map Switching Algorithm and Generation Pipeline for Autonomous Mobile Robots in Eccentric Scenarios

line 1: 1st Given Name
Surname
line 2: *dept. name of
organization (of
Affiliation)*
line 3: *name of
organization (of
Affiliation)*
line 4: City, Country
line 5: email address

line 1: 2nd Given Name
Surname
line 2: *dept. name of
organization (of
Affiliation)*
line 3: *name of
organization (of
Affiliation)*
line 4: City, Country
line 5: email address

line 1: 3rd Given Name
Surname
line 2: *dept. name of
organization (of
Affiliation)*
line 3: *name of
organization (of
Affiliation)*
line 4: City, Country
line 5: email address

line 1: 4th Given Name
Surname
line 2: *dept. name of
organization (of
Affiliation)*
line 3: *name of
organization (of
Affiliation)*
line 4: City, Country
line 5: email address

Abstract — Autonomous mobile robots operating in large indoor environments face a fundamental tension between map resolution and computational efficiency. Monolithic occupancy grid maps scale quadratically with environment size, imposing unsustainable memory and planning costs on resource-constrained embedded platforms. This paper presents a *tile-based* map switching system using *ROS2* and *Nav2* that decomposes an *occupancy grid* into spatially overlapping tiles and dynamically loads only the tile corresponding to the robot’s current location. A custom *Behavior Tree action node* with heading-aware trigger zones and a four-state asynchronous state machine enables seamless tile transitions without interrupting navigation. Validated on a differential drive robot in a 60 meters $\times$ 45 meters indoor corridor environment, benchmarks demonstrate 93.8 percent reduction in active costmap cells and 77.6 percent reduction in Nav2 Planner Server memory, at 11.9 percent mean CPU overhead concentrated at tile switch events.

Keywords — *tile-based map switching, large-scale navigation, occupancy grid, memory-efficient mapping, dynamic map management*

I. INTRODUCTION

Autonomous mobile robots operating in indoor environments depend fundamentally on accurate and computationally efficient map representations. The map serves as the primary spatial reference for localization, path planning, and obstacle avoidance, making its quality and management strategy central to overall navigation performance. As deployment environments expand from small rooms to building-scale spaces such as hospitals, warehouses, and university facilities, the demands placed on map representations have grown substantially.

The dominant paradigm for indoor robot navigation is the two-dimensional occupancy grid map [1]–[3], in which the environment is discretized into a uniform grid of cells, each assigned a probability of occupancy. Occupancy grids are intuitive, compatible with standard range sensors, and directly supported by widely used navigation frameworks such as ROS 2 and Nav2 [6]. However, their memory and computational cost scale with the square of the environment’s linear dimension at a fixed resolution. For a building-scale indoor environment spanning 60 meters $\times$ 45

meters at a navigation-grade resolution of 0.04 meters per cell, a single monolithic occupancy grid requires over 1.46 million cells, allocating, updating, and planning over this costmap imposes a significant burden on resource-constrained embedded platforms such as the *NVIDIA Jetson Nano*.

A broader survey of long-term localization and mapping strategies for mobile robots is provided in [4]. Several alternative representations have been proposed to address this scaling challenge. Three-dimensional map representations, including *OctoMap* [2] and *ColMap* [1], address vertical dimensionality through hierarchical octree and run-length encoding schemes, achieving order-of-magnitude memory reductions, *OctoMap* reports up to 97% memory reduction through pruning, while *ColMap* achieves 12 to 15 times lower memory than *OctoMap* at equivalent resolution. LiDAR-based frameworks such as SLAM2REF [7] extend this line of work by integrating long-term 3D reference maps for precise six-degree-of-freedom trajectory estimation, though at higher sensing and computational cost than 2D vision-based approaches. However, these methods still maintain a single global map whose memory footprint scales with environment size. Landmark-based maps [5] represent the environment as sparse feature sets, achieving mean localization accuracy of 0.284 meters on urban-scale datasets. *RC-Map* [8] partitions free space into rectangular regions, requiring only 1.5 percent of grid-based planning computation and 3 percent of grid memory for large environments.

A more targeted strategy is partitioning the environment into spatial submaps loaded and unloaded at runtime. Semantic zone-based map management [9] implements this for 3D maps in hospital environments using *RTAB-Map* with manually defined zones. Submap joining approaches [10] optimize global map consistency from local submaps, achieving path joining of environments up to 250 meters $\times$ 200 meters in 2 to 6 seconds. *Horelican* [6] evaluates Nav2 on a custom platform, identifying that high-resolution maps cause bandwidth overload regardless of other navigation parameters, directly motivating map coverage management. However, existing submap approaches either target 3D maps, require specialized *SLAM* (*Simultaneous*

Localization and Mapping) frameworks, or focus on map building rather than real-time navigation on pre-built maps.

The present work addresses this gap by proposing a tile-based map switching system for 2D occupancy grid navigation. The environment map is decomposed offline into spatially overlapping tiles, each covering one navigable zone of the floor. At runtime, only the tile corresponding to the robot's current location is loaded into the map server. A custom Behavior Tree action node monitors the robot's pose and heading at each navigation cycle and initiates an asynchronous tile switch when the robot enters a predefined overlap region between adjacent tiles.

TABLE I. COMPARISON OF MAP REPRESENTATION APPROACHES FOR INDOOR ROBOT NAVIGATION

Method	Representat ion	Memo ry	Partial Load	2D Nav2	Embedd ed
OctoMa p [2]	3D Octree	$O(n^3)$ compr.	No	No	Partial
olMap [1]	2D N^2 -tree +RLE	Sub- linear	No	No	Partial
OGM [3]	Binary 2D Grid	$O(n^2)$	No	Yes	Yes
RC- Map [8]	Rect. Graph	$\sim 3\%$ of grid	No	Custo m	Yes
Landma rk [5]	Sparse features	$O(\text{land-}$ $\text{marks})$	No	No	Yes
Semanti c [9]	3D RTAB +zones	Bound ed	Yes(zon e)	No	Partial
Submap Joining [10]	Grid submaps	$O(\text{sub-}$ $\text{maps})$	No	Partia l	Partial
Propose d	2D OGM tiles	$O(1)$ active	Yes (tile)	Yes	Yes

The proposed tile-based approach is the only method in the surveyed literature that simultaneously satisfies all five criteria. The key contributions are: (1) a tile-based decomposition strategy constraining active costmap size to a per-tile constant; (2) a heading-aware trigger zone mechanism for bidirectional transitions; (3) a non-blocking asynchronous state machine for seamless switching; and (4) a quantitative benchmark comparing monolithic and tessellated approaches.

The remainder of this paper is organized as follows. Section 2 describes the experimental platform and software stack. Section 3 presents the map data generation pipeline. Section 4 details the map switching algorithm. Section 5 presents benchmark results. Section 6 concludes.

II. EXPERIMENTAL SETUP AND SOFTWARE TOOLS

This section describes the robotic platform, sensor configuration, target deployment environment, and software infrastructure.

A. Robot Platform



Fig. 1. The NI Dani-based differential drive robot platform with custom superstructure. The ZED Mini stereo camera is mounted at 0.8 meters

height on the front face, tilted 30 degrees downward. The elevated acrylic tray (0.75 meters) serves as the material transfer payload.

The experimental platform is based on the NI Dani mobile robot chassis configured for differential drive operation, extended with a custom superstructure: four vertical carbon fiber tubes supporting an elevated acrylic payload tray at 0.75 meters. The onboard computing unit is an *NVIDIA Jetson Nano* (4 GB RAM, ARM64) running *Ubuntu 22.04* with *ROS2 Humble*. The motor driver is a *Cytron SmartDuo30* dual-channel brushed DC motor controller. The software stack is containerized using Docker for development consistency across AMD64 and ARM64 platforms.

B. Sensors and Localization

The robot is equipped with a single ZED Mini stereo camera (Stereolabs, SDK v4.2.5) as the sole exteroceptive sensor, mounted at 0.8 meters height on the front face, tilted 30 degrees downward. Localization is performed entirely by the *ZED SDK (Software Development Kit)* using visual-inertial odometry with IMU (Inertial Measurement Unit) fusion.

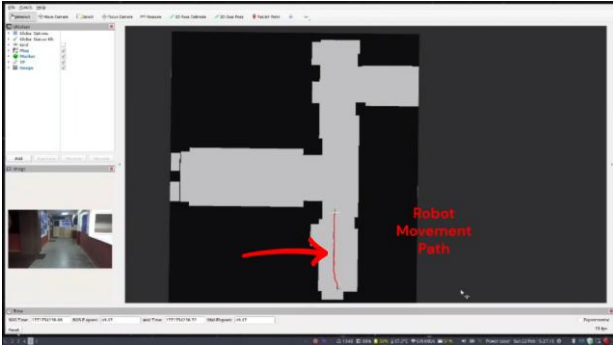


Fig. 2 RViz visualization of a tessellated tile loaded during live navigation. The active costmap covers only the current tile region; the remainder of the floor is not represented in memory.

C. Application and Deployment Environment

The target environment is a university building corridor network spanning approximately 60 meters $\times$ 45 meters with laboratories, classrooms, and staff rooms. At 0.04 meter per cell resolution, the monolithic map spans 1,466,724 cells, of which only 11.7 percent are navigable, the remaining 88.3 percent corresponds to walls and room interiors.

D. System Requirements

- a) 0.04 m/cell resolution
- b) tile switch latency < 500 ms
- c) 500 ms minimum inter-switch interval
- d) 300 ms post-switch costmap rebuild delay
- e) platform memory constrained by Jetson Nano 4 GB RAM.

E. Software Libraries and Tools

TABLE II SOFTWARE LIBRARIES AND TOOLS USED IN THE SYSTEM

Library / Tool	Version	Role
ROS 2 Humble	Humble	Middleware and navigation
Nav2	1.1.20	Navigation stack
BehaviorTree.CPP	3.8.7	BT execution engine
ZED SDK	4.2.5	Stereo camera, VIO
CycloneDDS	1.3.4	DDS middleware
yaml-cpp	8.0.2	Tile config parsing
tf2	0.25.16	Pose lookup
Docker	29.3.0	Containerized deployment
OpenCV	3.2.1	Image processing

III. MAP DATA GENERATION PIPELINE

This section describes the end-to-end pipeline for generating tessellated occupancy grid maps through five stages: physical measurement, CAD drawing, DXF export, pixel-to-metric calibration, occupancy annotation, and tessellation.

A. Physical Measurement

All corridor widths, room doorway positions, wall offsets, and junction dimensions were measured manually using a surveyor's tape at floor level. A reference distance of 2.3 meters was measured along a straight corridor section for subsequent resolution calibration.

B. CAD Drawing and DXF Export

Using the recorded measurements, a 2D sketch was constructed in SolidWorks capturing all navigable corridor regions, wall boundaries, and doorway positions. Room interiors are excluded.

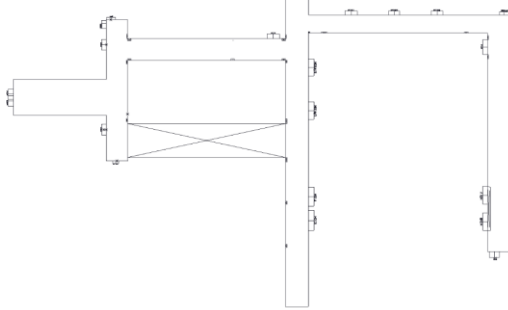


Fig. 3 SolidWorks CAD sketch (DXF export). Corridor outlines, wall boundaries, and room positions (H201–H219) are annotated.

C. Resolution Calibration

The map resolution was determined by pixel-to-metric calibration: a known physical distance of 2.3 meters corresponds to 55 pixels in the exported PNG, yielding

$$\text{Resolution} = 2.3 \text{ m} / 55 \text{ px} \approx 0.04 \text{ m/px}. \quad (1)$$

This value was adopted uniformly across all five tiles.

D. Occupancy Annotation

The grayscale PNG was annotated as a binary occupancy map: navigable regions painted white (free space) and non-navigable regions painted black (occupied). The annotated whole-floor map spans $1,596 \times 919$ pixels ($63.8 \text{ meters} \times 36.8 \text{ meters}$ at $0.04 \text{ meters per pixel}$).



Fig. 4 Full floor occupancy map after annotation. White = navigable corridor; black = occupied. Only 11.7 percent of cells are navigable, motivating tessellation.

E. Map Tessellation

The floor layout was divided into five tiles at natural turning points (T-junctions and L-junctions). At each junction, cut lines are placed with a 40-pixel (1.6 meters)

overlap offset, creating trigger zones for tile switches. Each tile covers one navigable zone of the corridor network.

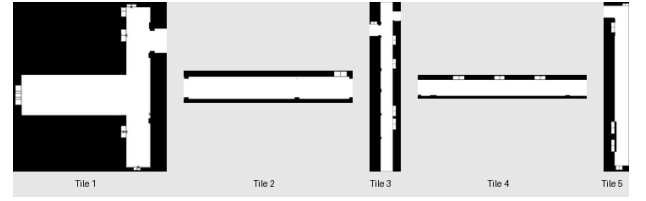


Fig. 5 Individual tessellated tile maps (Tile 1–5). Each tile covers one navigable zone. Tiles vary in shape to match corridor topology.

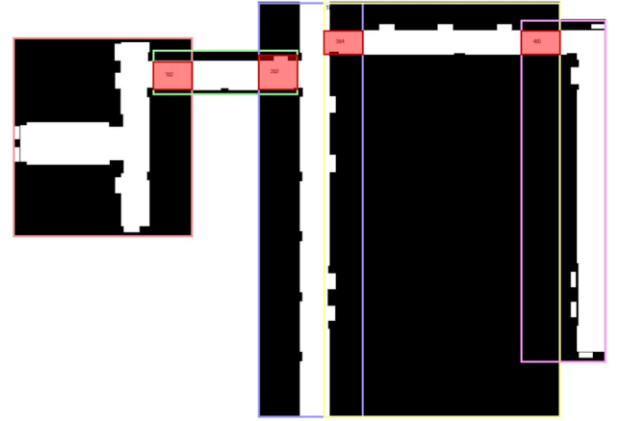


Fig. 6 Tessellated map tile layout with overlap regions. Colored bounding boxes indicate tile extents; red rectangles mark trigger zones for tile switches between adjacent pairs (1↔2, 2↔3, 3↔4, 4↔5).

TABLE III TILE DIMENSIONS AND NAVIGABLE CELL STATISTICS AT 0.04 M/PIX.

Tile	Width (meters)	Height (meters)	Cells (sq. pixels)	Navigable (percentage)
Tile 1	25.1	36.7	576,153	16.8%
Tile 2	19.5	28.3	345,272	12.1%
Tile 3	9.4	28.3	166,433	20.3%
Tile 4	9.4	20.5	120,797	18.7%
Tile 5	19.5	16.6	202,890	14.5%

IV. MAP SWITCHING ALGORITHM

The map switching algorithm is the primary contribution of this work. It enables a Nav2-based robot to navigate across a tessellated environment by dynamically loading and unloading map tiles at runtime without interrupting the navigation pipeline.

A. Integration with the Nav2 Behaviour Tree

TileCheckAction is implemented as a custom Behavior Tree action node (nav2_behavior_tree plugin) inserted into the standard Nav2 ReactiveSequence alongside FollowPath. It is ticked on every BT cycle and returns SUCCESS immediately to allow navigation to proceed concurrently with tile switching.

B. Tile Configuration

All tile definitions, spatial bounds, neighbour relationships, and trigger zone coordinates are specified in a centralized YAML configuration file (tiles_config.yaml) with three sections: settings (global parameters), tiles (per-tile definitions), and connections (adjacent tile pairs with trigger zones).

C. Switching Cues: Position and Heading

- a) *Spatial Containment*: The robot's current position (x, y) is obtained via *tf2 lookup* of the *map*→*base_link transform*. A trigger zone is an axis-aligned bounding box defined in world coordinates. When the robot's position falls within a trigger zone, the algorithm proceeds to heading verification.
- b) *Heading-Based Directional Disambiguation*: Spatial containment alone is insufficient: an overlap region is shared by exactly two tiles, and without heading information the algorithm cannot distinguish a robot transitioning from Tile A to Tile B from one transitioning from B to A. The robot's orientation (yaw) is compared against the expected travel direction stored in the tile connection definition. A switch is triggered only if the heading is within ± 90 degrees of the expected direction.

D. Asynchronous Switch State Machine

Once a valid trigger is detected, the algorithm initiates a tile switch through a four-state asynchronous state machine shown in Fig. 7. Asynchronous (non-blocking) service calls are critical: a blocking call would stall the Behavior Tree and halt navigation. The states are: *IDLE* (normal monitoring), *LOADING_MAP* (async LoadMap request to map_server), *CLEARING* (async costmap clear via ClearEntireCostmap), and *POST_DELAY* (300 ms wait for costmap rebuild). Navigation continues uninterrupted throughout all states.

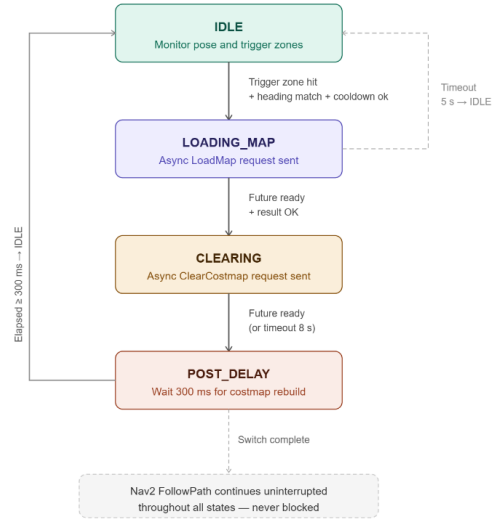


Fig. 7 Asynchronous tile switch state machine. States: *IDLE* (normal monitoring), *LOADING_MAP* (async LoadMap request), *CLEARING* (async costmap clear), *POST_DELAY* (wait for costmap rebuild). Navigation continues throughout all states.

E. Switching Artifacts and Mitigation

A known artifact is that the robot operates briefly on a stale *costmap* during the 100 to 300 milliseconds switch window, as *TileCheckAction* always returns *SUCCESS* and *FollowPath* continues execution. In practice, this is benign because the robot remains within the overlap region, which is valid on both tiles during the entire transition.

F. Demonstration

A video demonstration of the tile switching system operating in the live corridor environment is available as supplementary material [video link to be added after review]. The video shows the robot traversing across multiple tiles with seamless transitions.

V. RESULTS AND DISCUSSION

This section presents a quantitative comparison of monolithic and tessellated approaches across costmap size, Nav2 process memory, and CPU utilization. Benchmarks were conducted over a 120-second navigation run on an AMD64 platform.

A. Active Costmap Size

The most significant reduction is in active costmap cell count. The monolithic map maintains a fixed costmap of 1,466,724 cells. The tessellated approach reduces this to a mean of 91,594 cells (93.8 percent reduction). Peak tile size occurs at Tile 1 (96,741 cells, 16.8 percent navigable), still 93.4 percent smaller than the monolithic map.

B. Process-Level Memory Consumption

TABLE IV PER-PROCESS MEMORY CONSUMPTION: MONOLITHIC VS. TESSELLATED APPROACH (120 S RUN).

Process	Monolithic (MB)	Tessellated (MB)	Reduction
Nav2 Planner Server	154.5	34.6	77.6%
Map Server	12.8	2.6	79.7%
Nav2 Navigator	8.3	6.1	26.5%
Nav2 Controller	1.8	1.7	5.6%
Total (Nav2)	177.4	45.0	74.6%

The dominant contributor is the Nav2 Planner Server, dropping from 154.5 MB to 34.6 MB (77.6 percent reduction), consistent with the planner allocating data structures proportional to costmap size.

C. CPU Utilization

Mean CPU utilization was 32.8 percent (monolithic) versus 44.7 percent (tessellated), with peaks of 65.7% and 91.3% respectively. The higher CPU usage is attributed to tile switch events, confirming the memory-for-CPU tradeoff inherent in dynamic map management.

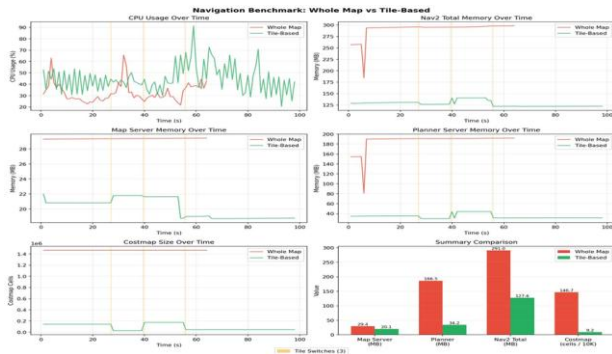


Fig. 8 Navigation benchmark: Whole Map vs. Tile-Based. Top row: CPU usage and Nav2 total memory over time. Middle row: Map Server and Planner Server memory. Bottom row: Costmap cell count and summary bar chart. Vertical yellow dashed lines indicate tile switch

D. Summary

TABLE V SUMMARY COMPARISON OF MONOLITHIC AND TESSELLATED APPROACHES.

Metric	Monolithic	Tessellated	Change
Active costmap cells	1,466,724	91,594 (mean)	−93.8%
Nav2 Planner memory	154.5 MB	34.6 MB	−77.6%
Map Server memory	12.8 MB	2.6 MB	−79.7%
Total Nav2 memory	177.4 MB	45.0 MB	−74.6%
Mean CPU utilization	32.8%	44.7%	+11.9 pp
Peak CPU utilization	65.7%	91.3%	+25.6 pp

The tessellated approach achieves substantial reductions in active costmap size and Nav2 planner memory, at the cost of moderate CPU overhead concentrated at tile switch events. The modular tile structure enables straightforward extension to additional floor zones.

VI. CONCLUSION

This paper presented a tile-based map switching system for autonomous mobile robot navigation in large indoor environments using the ROS 2 and Nav2 stack. The proposed approach decomposes a pre-built 2D occupancy grid into spatially overlapping tiles and dynamically loads only the active tile at runtime. A custom Behavior Tree action node with heading-aware trigger zone mechanism and four-state asynchronous state machine enables seamless tile transitions without interrupting navigation.

Benchmarks demonstrate a 93.8 percent reduction in active costmap cells and 77.6 percent reduction in Nav2 Planner Server memory, validating the approach for resource-constrained embedded platforms. The YAML-based tile configuration enables modular extension to additional zones without stack modification.

Future work includes: (1) automating the map generation pipeline to derive tessellation boundaries directly from CAD data; (2) adaptive trigger zone sizing based on robot velocity; (3) dynamic tile loading in multiple directions for multi-floor environments; and (4) tile-specific inflation layers for varying obstacle clearance requirements across zones.

ACKNOWLEDGMENT

[to be added after acceptance to ensure double blind compliance]

REFERENCES

- [1] A. Fisher, R. Cannizzaro, M. Cochrane, C. Nagahawatte, and J. L. Palmer, "ColMap: A memory-efficient occupancy grid mapping framework," *Robotics and Autonomous Systems*, vol. 142, p. 103755, 2021.
- [2] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss, and W. Burgard, "OctoMap: An efficient probabilistic 3D mapping framework based on octrees," *Autonomous Robots*, vol. 34, pp. 189–206, 2013.
- [3] D. Gonzalez-Arjona, A. Sanchez, F. López-Colino, A. de Castro, and J. Garrido, "Simplified occupancy grid indoor mapping optimized for low-cost robots," *ISPRS Int. J. Geo-Information*, vol. 12, no. 3, p. 96, 2023.
- [4] R. B. Sousa, H. M. Sobreira, and A. P. Moreira, "A systematic literature review on long-term localization and mapping for mobile robots," *Journal of Field Robotics*, vol. 40, pp. 1245–1322, 2023.
- [5] A. Schaefer, D. Büscher, J. Vertens, L. Luft, and W. Burgard, "Long-term vehicle localization in urban environments based on pole landmarks extracted from 3-D LiDAR scans," *Robotics and Autonomous Systems*, vol. 143, p. 103810, 2021.
- [6] T. Horelican, "Utilizability of Navigation2/ROS2 in highly automated and distributed multi-robotic systems for industrial facilities," *IFAC PapersOnLine*, vol. 55-4, pp. 109–114, 2022.
- [7] M. A. Vega-Torres, A. Braun, and A. Borrmann, "SLAM2REF: Advancing long-term mapping with 3D LiDAR and reference map integration for precise 6-DoF trajectory estimation and map extension," *Construction Robotics*, vol. 8, p. 6, 2024.
- [8] Y. Baek and J. K. Park, "Fast path generation algorithm for mobile robot navigation using hybrid map," *Applied Sciences*, vol. 15, p. 2414, 2025.
- [9] H. Yun and S. Yoo, "Semantic zone based 3D map management for mobile robot," *arXiv preprint arXiv:2512.12228*, Dec. 2025.
- [10] Y. Wang, L. Zhao, and S. Huang, "Grid-based submap joining: An efficient algorithm for simultaneously optimizing global occupancy map and local submap frames," *arXiv preprint arXiv:2501.12764*, Jan. 2025.